

KLAROS — System Architecture & Mathematical AHP Algorithm Guide

A Hybrid Decision Support Platform Combining Client-Side Operational Analytics, Multi-Provider LLM Orchestration, and the Deterministic Analytic Hierarchy Process (AHP).

Domain: Applied AI / Decision Intelligence **Core Algorithm:** Saaty Analytic Hierarchy Process (AHP)

Currency Standard: Indian Rupee (₹ INR) **Frameworks:** React 18 + Node 20 Serverless + Supabase

1 Executive Pitch: Explaining KLAROS in 2 Minutes

Verbal Pitch for Your Guide / Examiner:

"Respected Guide and Panel Members, most retail analytics tools are either purely descriptive dashboards (showing past charts without recommending actions) or unreliable black-box LLM chatbots (which hallucinate numbers, fail basic arithmetic, and make logically inconsistent decisions)."

KLAROS is a hybrid Decision Intelligence platform that decouples qualitative evaluation from arithmetic computation:

1. **Deterministic Analytics (Web Worker):** Ingests raw multi-format spreadsheets and calculates exact financial KPIs (revenues in ₹, margins %, stockout ratios) off the main thread.
2. **Qualitative AI Evaluator (LLM):** Evaluates business trade-offs and generates 1-to-1 pairwise comparisons on Saaty's 1–9 fundamental scale.
3. **Deterministic Math Engine (AHP):** Takes pairwise comparisons in TypeScript, builds positive reciprocal matrices, derives criteria weights via the **Geometric Mean Method**, verifies logical consistency via the **Consistency Ratio (CR < 0.10)**, and synthesizes 100-point ranked decision scores.

*This guarantees **zero arithmetic hallucinations**, **100% mathematical reproducibility**, and **provable logical consistency**."*

2 End-to-End System Architecture

The system operates across 4 coordinated tiers designed for low operating cost, zero client-key exposure in production, and high client responsiveness:

Architecture Tier	Component / File	Engineering Mechanics & Purpose
1. Ingestion & Normalization	document-extractor.ts schema-mapper.ts	Universal multi-format parser (CSV, XLSX, JSON, XML, TXT). Uses synonym dictionaries & fuzzy matching to normalize raw headers and auto-heal missing fields (e.g. deriving Total Revenue = Unit Price × Quantity in ₹).
2. Deterministic KPI Processing	market-metrics.worker.ts market-metrics-core.ts	Pure Web Worker engine computing gross margins, stock ratios, and inventory value off-thread. Uses Schwartzian transforms for O(n) chronological sorting.
3. AI Fallback Dispatcher	/api/llm (Vercel Node 20) llm-proxy-client.ts	Serverless proxy with zero-dependency RS256 Web Crypto Clerk JWT authentication. Multi-provider failover: Groq (Llama 3.3 70B) → OpenRouter (DeepSeek V4) → Gemini (2.5 Flash) with Upstash Redis caching (24h TTL).
4. AHP Mathematical Engine	ahp-math.ts analysis-schema.ts	Saaty scale clamping [1/9, 9], positive reciprocal matrix construction, Geometric Mean priority vectors, Principal Eigenvalue ($\lambda_{\max}$), and Consistency Ratio (CR < 0.10) validation.

3 Complete Mathematical AHP Walkthrough with Indian Rupee (₹) Example

To demonstrate the exact formulas, let us trace a real execution with a sample 3-product supermarket batch.

3.1 The Sample Supermarket Dataset

SKU	Product Name	Unit Price (P)	Unit Cost (C)	Daily Units Sold (Q)	Current Stock (S)	Reorder Point (R)	Operational Status
SKU-01	Fresh Milk 1L	₹60	₹50	100 units	20 units	80 units	Critical Stockout (S < R)
SKU-02	Gourmet Cookies	₹200	₹80	10 units	500 units	50 units	Severe Overstock (10x R)
SKU-03	Organic Rice 5kg	₹500	₹400	20 units	40 units	40 units	Balanced (S = R)

3.2 Deterministic Financial KPI Computation (TypeScript Web Worker)

1. Total Revenue = Σ (Unit Price × Quantity)

Revenue = (₹60 × 100) + (₹200 × 10) + (₹500 × 20) = ₹6,000 + ₹2,000 + ₹10,000 = ₹18,000

2. Total Cost = Σ (Unit Cost × Quantity)

Cost = (₹50 × 100) + (₹80 × 10) + (₹400 × 20) = ₹5,000 + ₹800 + ₹8,000 = ₹13,800

3. Net Profit & Gross Profit Margin %

Profit = ₹18,000 - ₹13,800 = ₹4,200
Margin % = (₹4,200 / ₹18,000) × 100 = 23.33%

4. Product Margins & Stockout Ratios (Stock / Reorder)

- Fresh Milk: Margin = (60 - 50)/60 = 16.67% | Stock Ratio = 20 / 80 = 0.25 (Critical Alert)
- Gourmet Cookies: Margin = (200 - 80)/200 = 60.00% | Stock Ratio = 500 / 50 = 10.0 (Overstocked)
- Organic Rice: Margin = (500 - 400)/500 = 20.00% | Stock Ratio = 40 / 40 = 1.00 (Balanced)

3.3 What the LLM Generates (Qualitative Strategy & Pairwise Comparisons)

The LLM receives the structured KPI summary and returns 3 Strategic Options, 3 Evaluation Criteria, and Saaty 1–9 Comparison Triples:

3 Strategic Alternatives (O):

- O1: Emergency Milk Restock & Supplier Reorder
- O2: Discount Promotion on Gourmet Cookies (Clear Stock)
- O3: Maintain Status Quo & Monitor Weekly Velocity

3 Evaluation Criteria (C):

- C1: Stockout & Revenue Risk Mitigation
- C2: Immediate Profit Margin Preservation
- C3: Working Capital & Holding Cost Liquidation

Criteria Comparison Triple: [C1 vs C2 = 3, C1 vs C3 = 2, C2 vs C3 = 1] (on Saaty's 1–9 Scale).

3.4 Building the Positive Reciprocal Criteria Matrix (M_C)

Given the comparison triple [a01 = 3, a02 = 2, a12 = 1], buildPairwiseMatrix() builds the 3×3 reciprocal matrix:

```
Matrix M_C = [ [1, c01, c02], [1/c01, 1, c12], [1/c02, 1/c12, 1] ]
[ 1.000, 3.000, 2.000 ]
[ 0.333, 1.000, 1.000 ]
[ 0.500, 1.000, 1.000 ]
```

3.5 Deriving Criteria Weights via Geometric Mean Method

In calculatePriorityVector(matrix):

```
Step 1: Compute Row Geometric Means: GM_i = (a_i1 * a_i2 * a_i3)^(1/3)
• Row 1 (C1: Stockout Risk): GM_1 = (1.0 * 3.0 * 2.0)^(1/3) = (6.0)^(1/3) = 1.8171
• Row 2 (C2: Margin Preservation): GM_2 = (0.3333 * 1.0 * 1.0)^(1/3) = (0.3333)^(1/3) = 0.6934
• Row 3 (C3: Capital Liquidation): GM_3 = (0.5000 * 1.0 * 1.0)^(1/3) = (0.5000)^(1/3) = 0.7937

Step 2: Sum Geometric Means & Normalize: w_i = GM_i / Σ GM_k
Sum Σ GM = 1.8171 + 0.6934 + 0.7937 = 3.3042

• Weight w_1 (C1): 1.8171 / 3.3042 = 0.5499 (55.0%)
• Weight w_2 (C2): 0.6934 / 3.3042 = 0.2098 (21.0%)
• Weight w_3 (C3): 0.7937 / 3.3042 = 0.2402 (24.0%)
Σ w_i = 0.5499 + 0.2098 + 0.2402 = 1.0000 (Exact 100%)
```

3.6 Principal Eigenvalue (λ_max) & Consistency Index (CI)

```
Step 1: Compute Matrix-Vector Product: y = M_C · w
• y_1 = (1.0 * 0.5499) + (3.0 * 0.2098) + (2.0 * 0.2402) = 0.5499 + 0.6294 + 0.4804 = 1.6597
• y_2 = (0.3333 * 0.5499) + (1.0 * 0.2098) + (1.0 * 0.2402) = 0.1833 + 0.2098 + 0.2402 = 0.6333
• y_3 = (0.5000 * 0.5499) + (1.0 * 0.2098) + (1.0 * 0.2402) = 0.2750 + 0.2098 + 0.2402 = 0.7250

Step 2: Compute λ_i = y_i / w_i and Average λ_max
• λ_1 = 1.6597 / 0.5499 = 3.0182 | λ_2 = 0.6333 / 0.2098 = 3.0186 | λ_3 = 0.7250 / 0.2402 = 3.0183
λ_max = (3.0182 + 3.0186 + 3.0183) / 3 = 3.0184

Step 3: Compute Consistency Index: CI = (λ_max - n) / (n - 1)
CI = (3.0184 - 3) / (3 - 1) = 0.0184 / 2 = 0.0092
```

3.7 Consistency Ratio (CR) Validation

✓ Consistency Ratio Verification (CR < 0.10)

Using Saaty's empirical Random Index for n = 3 (RI_3 = 0.58):

CR = CI / RI_3 = 0.0092 / 0.58 = 0.0159 (1.59%)

Since CR = 0.0159 < 0.10, the qualitative comparisons are mathematically consistent, transitive, and verified!

3.8 Global Synthesis & Final 100-Point Composite Scores

Following the same geometric mean steps, the system computes the option priority vectors (v^(k)) for each criterion:

Criterion (C _k)	Weight (w _k)	Option 1 (O1: Restock Milk)	Option 2 (O2: Promo Cookies)	Option 3 (O3: Status Quo)
C1 (Stockout Risk)	0.550	v1 = 0.731	v2 = 0.188	v3 = 0.081
C2 (Margin Preservation)	0.210	v1 = 0.217	v2 = 0.550	v3 = 0.233
C3 (Capital Liquidation)	0.240	v1 = 0.122	v2 = 0.648	v3 = 0.230

Composite Score Formula: $S_j = \sum (w_k \times v_{jk}) \times 100$

- **Score 01:** $[(0.550 \times 0.731) + (0.210 \times 0.217) + (0.240 \times 0.122)] \times 100 = (0.4021 + 0.0456 + 0.0293) \times 100 = 48 / 100$
- **Score 02:** $[(0.550 \times 0.188) + (0.210 \times 0.550) + (0.240 \times 0.648)] \times 100 = (0.1034 + 0.1155 + 0.1552) \times 100 = 37 / 100$
- **Score 03:** $[(0.550 \times 0.081) + (0.210 \times 0.233) + (0.240 \times 0.230)] \times 100 = (0.0446 + 0.0489 + 0.0552) \times 100 = 15 / 100$

Rank 1 (Top Recommendation): Option 1 — Emergency Milk Restock (Score: 48/100)

Heavily favored because Stockout & Revenue Risk Mitigation carries 55% of the total criteria weight.

Rank 2: Option 2 — Discount Promotion on Gourmet Cookies (Score: 37/100)

Dominates on Working Capital Liquidation (v=0.648) and Margin Preservation (v=0.550).

Rank 3: Option 3 — Status Quo (Score: 15/100)

Lowest priority due to compounding stockout penalties on primary revenue drivers.

4 Zod Schema Hardening & Runtime Validation

In `src/features/decisions/core/analysis-schema.ts`, Zod serves as a strict runtime guard ensuring that malformed LLM outputs never crash the downstream mathematical engine:

```
// 1. Saaty Scale Constraint: bounds all pairwise comparisons to [1/9, 9]
const SaatyValueSchema = z
  .number()
  .min(1 / 9, 'Saaty value must be ≥ 1/9')
  .max(9, 'Saaty value must be ≤ 9')
  .refine((n) => Number.isFinite(n), 'Must be finite number');

export const ComparisonTripleSchema = z.tuple([
  SaatyValueSchema,
  SaatyValueSchema,
  SaatyValueSchema,
]);

// 2. Strict MCDA Response Schema
export const McdaRawResponseSchema = z.object({
  options: z.array(z.object({ id: z.string(), label: z.string() })).min(3).max(3),
  criteria: z.array(z.object({ id: z.string(), name: z.string() })).min(3).max(3),
  criteriaComparisons: ComparisonTripleSchema,
  optionComparisons: z.array(ComparisonTripleSchema).min(3).max(3),
  confidence: z.number().int().min(0).max(100),
  recommendation: z.string().min(10),
  reasoning: z.object({
    decomposition: z.string(),
    assumptions: z.array(z.string()),
    tradeoffs: z.array(z.string()),
    risks: z.array(z.string()),
    sensitivity: z.string(),
  }),
});
```

5 Resilient 5-Stage JSON Extraction Waterfall

In `src/services/llm/core/json-extractor.ts`, raw LLM strings pass through 5 sequential extraction strategies:

Stage	Extraction Strategy	Defect Repaired & Behavior
TRY 1	Direct <code>JSON.parse()</code>	Executes fast path when the model outputs clean, raw JSON.
TRY 2	Regex Code-Block Extractor	Extracts content between markdown code fences and repairs trailing commas.
TRY 3	Backtick Stripping Pass	Strips conversational markdown backticks and reparses.
TRY 4	Depth Bracket State Machine	Tracks brace/bracket depth { } [] while correctly ignoring quotes inside strings.
TRY 5	Bounded Tail-Trim ($O(n)$)	Trims up to 200 trailing characters to close truncated responses without quadratic overhead.

6 Multi-Provider LLM Fallback & Retry Protocol

To guarantee high availability against provider rate limits (HTTP 429) and quota exhaustion, KLAROS cascades through a prioritized provider chain:

Provider Priority Cascade:

1. Groq (Primary / Ultra-Fast): llama-3.3-70b-versatile → llama-3.1-8b-instant → qwen-3-32b

2. OpenRouter (Secondary / Diversity): deepseek-v4-flash → qwen3-32b → llama-4-scout

3. Google Gemini (Tertiary Fallback): gemini-2.5-flash → gemini-2.0-flash → gemini-1.5-flash

Bounded Exponential Backoff Formula:

$\text{Delay}(\text{attempt}) = \min(4000, 1000 \times 2^{\text{attempt}} + \text{jitter})$ ms across 3 retry attempts.

7 Zero-Trust Security & Database RLS Architecture

Serverless RS256 Web Crypto Auth

The `/api/llm` proxy function uses Node 20's native `crypto.subtle` to verify the Clerk-issued RS256 JWT signature using Clerk's public PEM key before making any external AI API call.

- Zero dependency bundle overhead (no `jsonwebtoken` package).
- Eliminates cold-start latency on Vercel Edge.
- Secret API keys stay strictly in encrypted Vercel environment variables.

PostgreSQL Row-Level Security (RLS)

Supabase enforces strict isolation matching the authenticated Clerk token claim against the table's `user_id` column:

```
CREATE POLICY "Users manage own decisions"
ON decisions FOR ALL
USING (auth.jwt() ->> 'sub' = user_id)
WITH CHECK (auth.jwt() ->> 'sub' = user_id);
```

8 Viva Presentation Q&A Cheat Sheet

Q1: Why did you choose AHP instead of Simple Additive Weighting (SAW)?

"SAW requires arbitrary percentage assignments which are cognitively biased. AHP decomposes evaluations into 1-to-1 pairwise comparisons, computes mathematically exact weights via the Geometric Mean method, and programmatically validates judgment consistency using the Consistency Ratio ($CR < 0.10$)."

Q2: How do you prevent division by zero or NaN errors in AHP?

"Our `clampSaaty()` utility clamps all comparison values to $[1/9, 9]$ and maps non-finite values to 1.0 (equal importance). In `calculatePriorityVector()`, if the geometric mean sum is zero, it safely falls back to a uniform distribution $[1/n, \dots, 1/n]$."

Q3: How do you ensure high performance during 50MB CSV uploads?

"All dataset metric calculations run off-thread in a dedicated Web Worker (`market-metrics.worker.ts`). Sorting uses Schwartzian transforms to avoid $O(n \log n)$ Date object allocations, ensuring the React UI remains at 60 FPS."

Q4: Is the revenue forecast generated by an ML model or LLM?

"It is generated via structured in-context LLM time-series extrapolation using the last 30 daily data points. It is not an ARIMA or Prophet statistical model, which we have transparently documented in our limitations."